Name : Vaishnavi Eknath Avhad
Class : D20C
Roll No. : 2

Practical 7

Aim : Introduction to Truffle, Establish local blockchain using Truffle and deployment of Smart Contract.

Theory

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment. Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Steps to Connect Ganache with Metamask and Remix IDE:

1. Install Ganache:
    a. Download and install Ganache from the Truffle Suite website.
    b. Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
    a. Open Metamask and switch to a Custom RPC Network.
    b. Enter the Ganache RPC server URL (e.g., HTTP://127.0.0.1:7545).
    c. Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
    a. Open Remix IDE in the browser.
4. Choose Injected Web3 as the environment in the “Deploy & Run Transactions” tab.
    a. Remix will automatically connect to the Metamask account (linked with Ganache).
5. Create a Simple Solidity Smart Contract:
    a. Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).
6. Compile and Deploy the Contract:
    a. Compile using Remix.
    b. Deploy via the Metamask account connected to Ganache.
    c. Approve the transaction in Metamask.
7. Check Transaction Details on Ganache:
    a. View mined transactions, block details, and balances in the Ganache interface.
8. Interact with the Smart Contract:
    a. Use Remix to call contract functions (set() or get()).

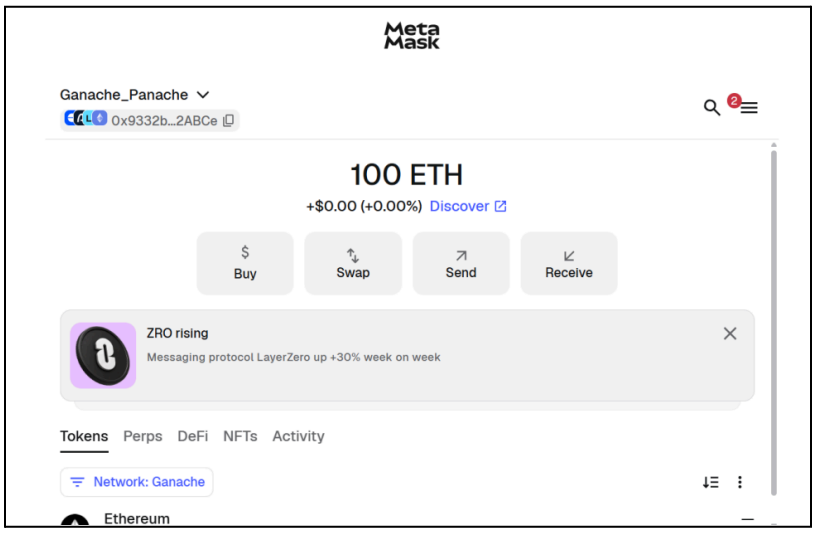
Observe the state changes and corresponding transactions on Ganache.

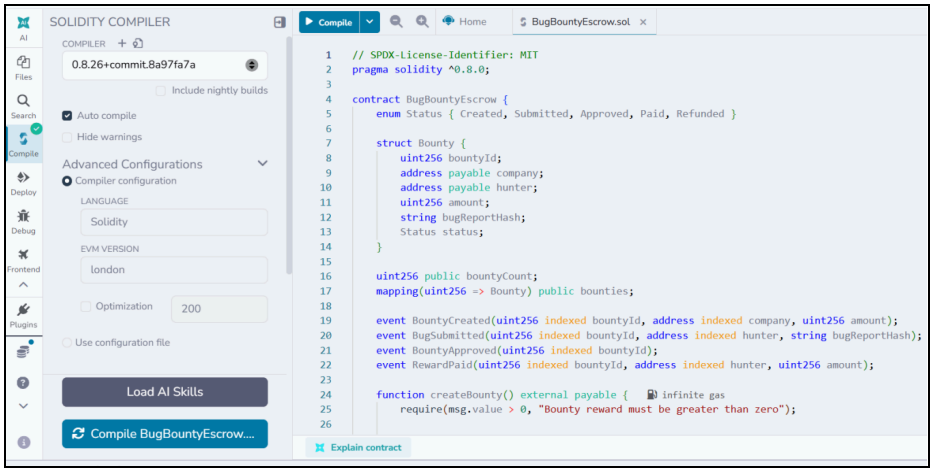
Program and Output

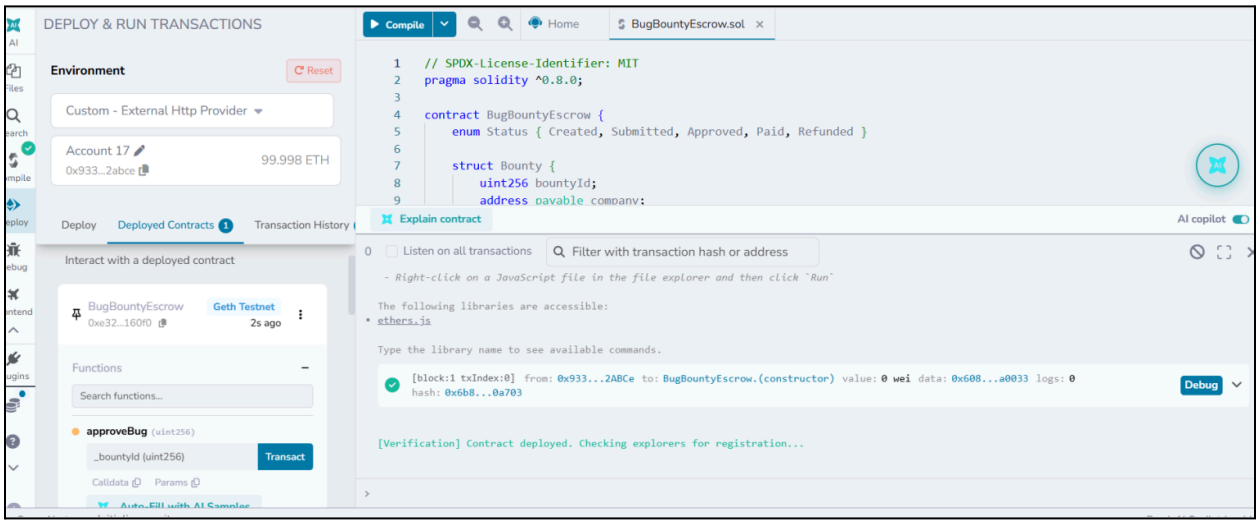
Code:

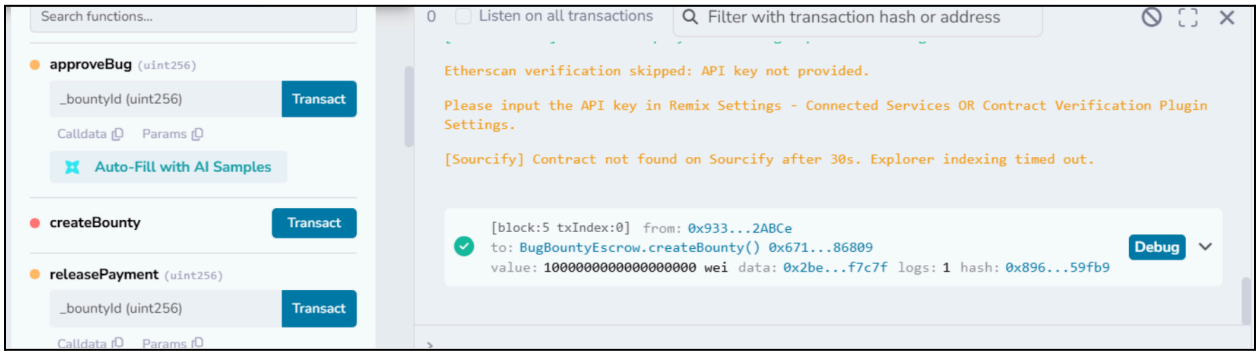
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract BugBountyEscrow {
    enum Status {
        Created,
        Submitted,
        Approved,
        Paid,
        Refunded
    }
    struct Bounty {
        uint256 bountyId;
        address payable company;
        address payable hunter;
        uint256 amount;
        string bugReportHash;
        Status status;
    }
    uint256 public bountyCount;
    mapping(uint256 => Bounty) public bounties;
    event BountyCreated(
        uint256 indexed bountyId,
        address indexed company,
        uint256 amount
    );
    event BugSubmitted(
        uint256 indexed bountyId,
        address indexed hunter,
        string bugReportHash
    );
    event BountyApproved(uint256 indexed
bountyId);
    event RewardPaid(
        uint256 indexed bountyId,
        address indexed hunter,
        uint256 amount
    );
    function createBounty() external payable {
        require(msg.value > 0, "Bounty reward
must be greater than zero");
        bountyCount++;
        bounties[bountyCount] = Bounty({
            bountyId: bountyCount,
            company: payable(msg.sender),
            hunter: payable(address(0)),
            amount: msg.value,
            bugReportHash: "",
            status: Status.Created
        });
        emit BountyCreated(bountyCount,
msg.sender, msg.value);
    }
    function submitBug(
        uint256 _bountyId,
        string memory _bugReportHash
    ) external {
        Bounty storage bounty =
bounties[_bountyId];
        require(bounty.bountyId != 0, "Bounty does
not exist");
        require(
            bounty.status == Status.Created,
            "Bounty is not open for submission"
        );
        bounty.hunter = payable(msg.sender);
        bounty.bugReportHash = _bugReportHash;
        bounty.status = Status.Submitted;
        emit BugSubmitted(_bountyId, msg.sender,
_bugReportHash);
    }
    function approveBug(uint256 _bountyId)
external {
        Bounty storage bounty =
bounties[_bountyId];
        require(msg.sender == bounty.company,
"Only company can approve");
        require(
            bounty.status == Status.Submitted,
            "No submission to approve"
        );
        bounty.status = Status.Approved;
        emit BountyApproved(_bountyId);
```

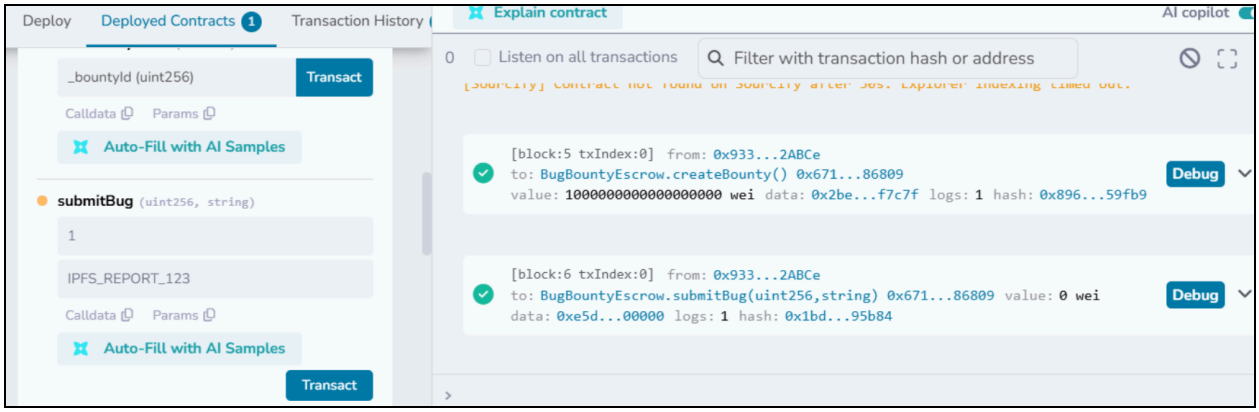
```
    }
    function releasePayment(uint256 _bountyId)
external {
        Bounty storage bounty =
bounties[_bountyId];
        require(
            msg.sender == bounty.company,
            "Only company can release payment"
        );
        require(
            bounty.status == Status.Approved,
            "Bounty report not approved yet"
        );
        bounty.status = Status.Paid;
        uint256 paymentAmount = bounty.amount;
        bounty.amount = 0;
        bounty.hunter.transfer(paymentAmount);
        emit RewardPaid(_bountyId, bounty.hunter,
paymentAmount);
    }
    function getBounty(uint256 _bountyId)
        external
        view
        returns (
            uint256 bountyId,
            address company,
            address hunter,
            uint256 amount,
            string memory bugReportHash,
            Status status
        )
    {
        Bounty memory b = bounties[_bountyId];
        require(b.bountyId != 0, "Bounty does not
exist");
        return (
            b.bountyId,
            b.company,
            b.hunter,
            b.amount,
            b.bugReportHash,
            b.status
        );
    }
}
```



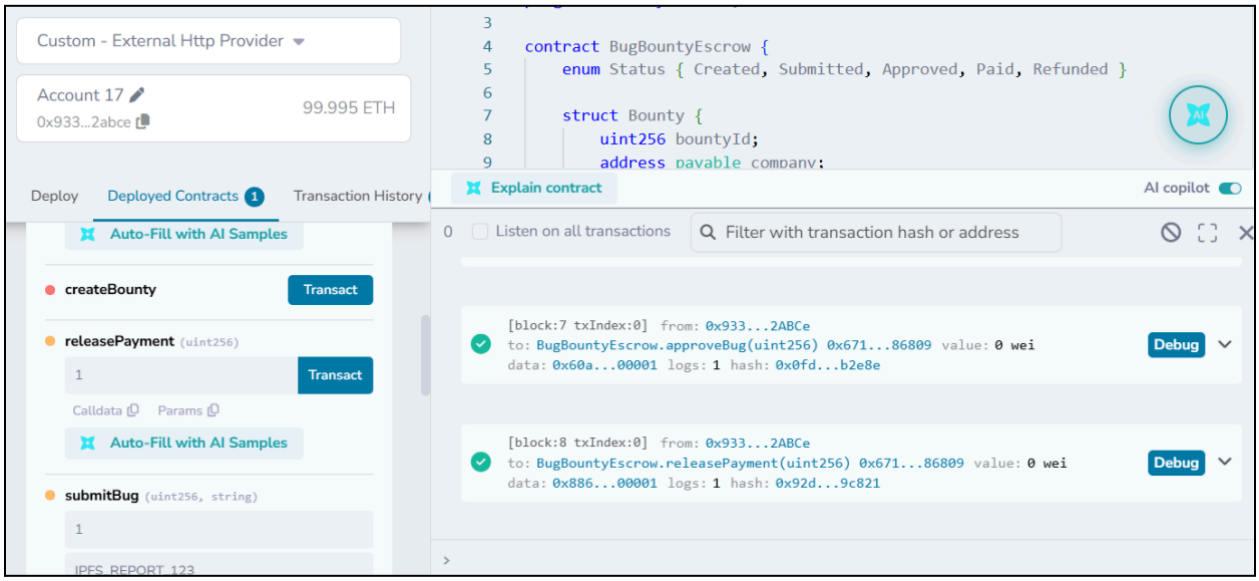

SOLIDITY COMPILER
COMPILER
0.8.26+commit.8a97fa7a
Include nightly builds
Auto compile
Hide warnings
Advanced Configurations
Compiler configuration
LANGUAGE
Solidity
EVM VERSION
london
Optimization
200
Use configuration file
Load AI Skills
Compile BugBountyEscrow....
Compile
Home
BugBountyEscrow.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract BugBountyEscrow {
enum Status { Created, Submitted, Approved, Paid, Refunded }
struct Bounty {
uint256 bountyId;
address payable company;
address payable hunter;
uint256 amount;
string bugReportHash;
Status status;
}
uint256 public bountyCount;
mapping(uint256 => Bounty) public bounties;
event BountyCreated(uint256 indexed bountyId, address indexed company, uint256 amount);
event BugSubmitted(uint256 indexed bountyId, address indexed hunter, string bugReportHash);
event BountyApproved(uint256 indexed bountyId);
event RewardPaid(uint256 indexed bountyId, address indexed hunter, uint256 amount);
function createBounty() external payable {
infinite gas
require(msg.value > 0, "Bounty reward must be greater than zero");
Explain contract

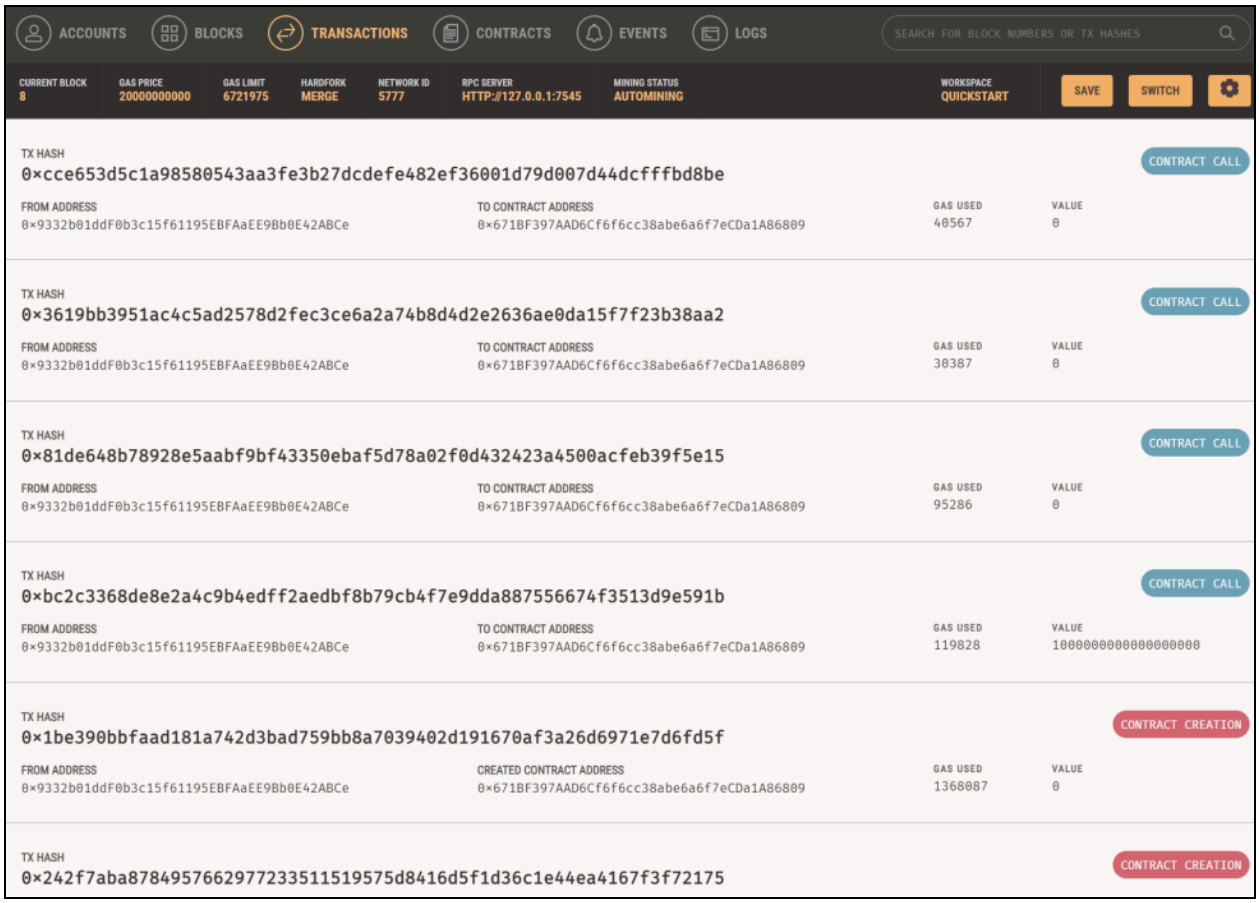

DEPLOY & RUN TRANSACTIONS
Environment
Reset
Custom - External Http Provider
Account 17
0x933...2abce
99.998 ETH
Deploy
Deployed Contracts 1
Transaction History
Interact with a deployed contract
BugBountyEscrow
0xe32...160f0
Geth Testnet
2s ago
Functions
Search functions...
approveBug (uint256)
_bountyId (uint256)
Transact
Calldata
Params
Compile
Home
BugBountyEscrow.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract BugBountyEscrow {
enum Status { Created, Submitted, Approved, Paid, Refunded }
struct Bounty {
uint256 bountyId;
address payable company;
Explain contract
AI copilot
0
Listen on all transactions
Filter with transaction hash or address
- Right-click on a JavaScript file in the file explorer and then click `Run`
The following libraries are accessible:
ethers.js
Type the library name to see available commands.
[block:1 txIndex:0] from: 0x933...2ABCe to: BugBountyEscrow.(constructor) value: 0 wei data: 0x608...a0033 logs: 0
hash: 0x6b8...0a703
Debug
[Verification] Contract deployed. Checking explorers for registration...


Search functions...
approveBug (uint256)
_bountyId (uint256)
Transact
Calldata
Params
Auto-Fill with AI Samples
createBounty
Transact
releasePayment (uint256)
_bountyId (uint256)
Transact
Calldata
Params
0
Listen on all transactions
Filter with transaction hash or address
Etherscan verification skipped: API key not provided.
Please input the API key in Remix Settings - Connected Services OR Contract Verification Plugin Settings.
[Sourcify] Contract not found on Sourcify after 30s. Explorer indexing timed out.
[block:5 txIndex:0] from: 0x933...2ABCe
to: BugBountyEscrow.createBounty() 0x671...86809
value: 1000000000000000000 wei data: 0x2be...f7c7f logs: 1 hash: 0x896...59fb9
Debug


Deploy
Deployed Contracts 1
Transaction History
Explain contract
AI copilot
_bountyId (uint256)
Transact
Calldata
Params
Auto-Fill with AI Samples
submitBug (uint256, string)
1
IPFS_REPORT_123
Calldata
Params
Auto-Fill with AI Samples
Transact
0
Listen on all transactions
Filter with transaction hash or address
[block:5 txIndex:0] from: 0x933...2ABCe
to: BugBountyEscrow.createBounty() 0x671...86809
value: 100000000000000000 wei data: 0x2be...f7c7f logs: 1 hash: 0x896...59fb9
Debug
[block:6 txIndex:0] from: 0x933...2ABCe
to: BugBountyEscrow.submitBug(uint256,string) 0x671...86809 value: 0 wei
data: 0xe5d...00000 logs: 1 hash: 0x1bd...95b84
Debug


Functions
Search functions...
approveBug (uint256)
1
Transact
Calldata
Params
Auto-Fill with AI Samples
[block:6 txIndex:0] from: 0x933...2ABCe
to: BugBountyEscrow.submitBug(uint256,string) 0x671...86809 value: 0 wei
data: 0xe5d...00000 logs: 1 hash: 0x1bd...95b84
Debug
[block:7 txIndex:0] from: 0x933...2ABCe
to: BugBountyEscrow.approveBug(uint256) 0x671...86809 value: 0 wei
data: 0x60a...00001 logs: 1 hash: 0x0fd...b2e8e
Debug


Custom - External Http Provider
Account 17
0x933...2abce
99.995 ETH
contract BugBountyEscrow {
enum Status { Created, Submitted, Approved, Paid, Refunded }
struct Bounty {
uint256 bountyId;
address payable company;
Deploy
Deployed Contracts 1
Transaction History
Explain contract
AI copilot
Auto-Fill with AI Samples
createBounty
Transact
releasePayment (uint256)
1
Transact
Calldata
Params
Auto-Fill with AI Samples
submitBug (uint256, string)
1
IPFS_REPORT_123
0
Listen on all transactions
Filter with transaction hash or address
[block:7 txIndex:0] from: 0x933...2ABCe
to: BugBountyEscrow.approveBug(uint256) 0x671...86809 value: 0 wei
data: 0x60a...00001 logs: 1 hash: 0x0fd...b2e8e
Debug
[block:8 txIndex:0] from: 0x933...2ABCe
to: BugBountyEscrow.releasePayment(uint256) 0x671...86809 value: 0 wei
data: 0x886...00001 logs: 1 hash: 0x92d...9c821
Debug



Conclusion

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with Metamask and Remix, developers can deploy, test, and debug smart contracts efficiently before moving to a public network.